claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a7a87af8403bda9e8.jsonl`
- SHA-256: `742991fff103a1f26cfd50a5521ebc83290adea45d10e468dc3bd143b005397e`
- Source modified: `2026-05-31T04:26:19+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:25:02.138Z)

Adversarial Claim Verifier (voter 1/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.

- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly.

Claim under review

"PP-StructureV3's lightweight configuration runs with roughly 7.4 GB peak VRAM (7456 MB) at ~0.61s average latency per page on an A100, making the full layout+table pipeline feasible on consumer 8-12 GB NVIDIA GPUs."

Source: https://github.com/PaddlePaddle/PaddleOCR/blob/main/docs/version3.x/algorithm/PP-StructureV3/PP-StructureV3.en.md (primary)
Supporting quote: "PP_StructureV3-lightweight on Intel 8350C + A100: Peak VRAM 7456 MB, Avg VRAM 7131.95 MB, Avg latency 0.61s"

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

refuted=true if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.
refuted=false ONLY if: claim is well-supported, current, and source quality matches claim strength.
Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-05-31T04:25:05.015Z)

I'll verify this claim about PP-StructureV3's lightweight configuration VRAM and latency. Let me start by examining the primary source and searching for corroborating/contradicting evidence.

3. assistant (2026-05-31T04:25:08.203Z)

I have my web tools loaded. Let me fetch the primary source and search for contradicting evidence in parallel.

4. user (2026-05-31T04:25:16.176Z)

Web search results for query: "PP-StructureV3 lightweight VRAM 7456 MB latency A100 benchmark"

Links: [{"title":"PP-StructureV3 Introduction - PaddleOCR Documentation","url":"https://paddlepaddle.github.io/PaddleOCR/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html"},{"title":"A100 GPU's Offer Power, Performance, & Efficient Scalability","url":"https://www.nvidia.com/en-us/data-center/a100/"},{"title":"NVIDIA A100 TENSOR CORE GPU | DATA SHEET | 1","url":"https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf"},{"title":"A100 40GB vLLM Benchmark: Cheap GPU for Gemma3 Hosting","url":"https://www.databasemart.com/blog/vllm-gpu-benchmark-a100-40gb"},{"title":"NVIDIA A100 CUDA Performance Guide: Specs, Benchmarks & Optimization | RightNow AI | RightNow AI","url":"https://www.rightnowai.co/guides/gpu-comparison/a100"},{"title":"NVIDIA A100 TENSOR CORE GPU | DATA SHEET | Jun21 | 1","url":"https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf"},{"title":"Data Center Deep Learning Product Performance Hub | NVIDIA Developer","url":"https://developer.nvidia.com/deep-learning-performance-training-inference"},{"title":"NVIDIA GPU benchmark: H100 vs A100 vs L40S","url":"https://www.cudocompute.com/blog/real-world-gpu-benchmarks"}]

The search results show information about PP-StructureV3 and A100 GPUs, but do not contain specific benchmark data matching your exact query parameters (7456 MB VRAM, latency measurements).

Based on the search results, PP-StructureV3 performance testing was conducted on both V100 and A100 GPU environments, and serving inference testing was performed on NVIDIA A100 + Intel Xeon Platinum 8350C environment. However, the detailed benchmark metrics with the specific VRAM usage and latency figures you mentioned (7456 MB) are not visible in these search results.

For more detailed benchmark specifications, you may want to consult the official PaddleOCR documentation or the specific benchmark report that contains these exact performance metrics.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

5. user (2026-05-31T04:25:18.612Z)

PP-StructureV3 Lightweight Configuration Performance Metrics

Based on the pipeline benchmark data table, here are the specifications for the **PP_StructureV3-lightweight** configuration:

Peak VRAM and Average VRAM Usage

Hardware	Peak VRAM	Avg. VRAM
Intel 8350C + A100	7,456 MB	7,131.95 MB
Intel 6271C + V100	8,450 MB	8,083.30 MB
Intel 8563C + H20	7,524 MB	7,167.52 MB
Intel 8350C + A10	7,248 MB	6,927.58 MB
Intel 6271C + T4	8,218 MB	7,902.04 MB

Average Latency (per page/image)

Ranging from **0.46 seconds** (Intel 8563C + H20) to **1.13 seconds** (Intel 6271C + T4).

Configuration Details

The document states: "all task models are replaced with lightweight versions." However, specific details about precision (e.g., FP32 vs. INT8), batch size, or single vs. multi-GPU operation are **not provided** in the benchmark methodology section.

The test used "280 images including tables, seals, formulas, and charts" across a single warm-up

plus one full dataset iteration.

6. assistant (2026-05-31T04:25:25.845Z)

The primary source confirms the raw numbers. Now I need to scrutinize the inferential leap in the claim: the numbers were measured on an A100 (a 40/80 GB datacenter GPU), and the claim asserts this makes it "feasible on consumer 8-12 GB NVIDIA GPUs." Let me check the source more carefully for the latency figure and verify the A100-specific latency, and probe whether 7.4 GB peak actually fits in an 8 GB card in practice.

7. user (2026-05-31T04:25:29.683Z)

REDIRECT DETECTED: The URL redirects to a different host.

Original URL: https://paddlepaddle.github.io/PaddleOCR/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html
Redirect URL: http://www.paddleocr.ai/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html
Status: 301 Moved Permanently

To complete your request, I need to fetch content from the redirected URL. Please use WebFetch again with these parameters:
- url: "http://www.paddleocr.ai/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html"
- prompt: "In the pipeline benchmark / performance table, for the PP_StructureV3-lightweight configuration on the "Intel 8350C + A100" hardware row, what is the exact average latency in seconds per page/image? Also report the peak VRAM for that A100 row. List every column in that row verbatim (CPU, GPU, peak VRAM, avg VRAM, avg latency, and any per-stage latency columns). Also state what precision/data type the benchmark used (FP32/FP16/INT8) if mentioned anywhere on the page, and the number of GPUs used."

8. user (2026-05-31T04:25:35.193Z)

Web search results for query: "PP-StructureV3 GPU memory requirement consumer GPU 8GB out of memory install"

Links: [{"title":"PP-StructureV3 Introduction - PaddleOCR Documentation","url":"https://paddlepaddle.github.io/PaddleOCR/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html"},{"title":"llama.cpp VRAM Requirements: Complete 2026 Guide to GPU Memory for Local LLMs | LocalLLM.in","url":"https://localllm.in/blog/llamacpp-vram-requirements-for-local-llms"},{"title":"Placement of virtual GPU requests in virtual GPU enabled systems using a requested memory requirement of the virtual GPU request","url":"https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/11372683"},{"title":"PIPO: Pipelined Offloading for Efficient Inference on Consumer Devices","url":"https://arxiv.org/pdf/2504.03664"},{"title":"Guide to GPU Requirements for Running AI Models","url":"https://www.bacloud.com/en/blog/163/guide-to-gpu-requirements-for-running-ai-models.html"},{"title":"GPU Memory Requirements for LLMs: VRAM Calculator | Spheron Blog","url":"https://www.spheron.network/blog/gpu-memory-requirements-llm/"}]

Based on the search results, I found relevant information about PP-StructureV3 GPU memory requirements:

By default, PP-StructureV3 uses the largest models for each module, and different model selections affect GPU memory consumption and inference speed. You can choose an appropriate model based on your device capabilities and the complexity of your samples.

While PP-StructureV3 is recommended to run on GPU for optimal performance, it also supports CPU inference. Thanks to a variety of configuration options and sufficient optimization for lightweight models, users can select lightweight configurations for CPU-only environments.

The official PaddleOCR documentation indicates that the memory requirements are flexible and depend on which model variant you select. If you're experiencing out-of-memory errors with an 8GB GPU, the solution would be to select a lighter-weight model configuration rather than the default largest models. The documentation references "section 3.3" which demonstrates how different model selections affect memory consumption and inference speed, suggesting that

downgrading to smaller models would be an appropriate solution for consumer GPUs with limited memory.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. assistant (2026-05-31T04:25:38.155Z)

Let me fetch the redirected URL to confirm the exact A100 latency and precision details, and search for real-world reports of PP-StructureV3 actual VRAM consumption (the benchmark "peak" may differ from real-world peak due to model loading, framework overhead, CUDA context, etc.).

10. user (2026-05-31T04:25:45.409Z)

PP-StructureV3 Benchmark Data

For the **PP_StructureV3-lightweight** configuration on Intel 8350C + A100 hardware, here is the complete row from the pipeline benchmark table:

All columns verbatim:
Intel 8350C + A100 | 0.61 | 823.40 | 126.25 | 9258.22 | 3776.63 | 52 | 18.95 | 7456.00 | 7131.95

- **Average inference time:** 0.61 seconds
- **Peak VRAM:** 7456.00 MB

Benchmark Methodology

The documentation states: "Warm up with 20 samples, then repeat the full dataset once for performance testing." The test used "280 images including tables, seals, formulas, and charts."

Precision/Data Type: Not specified in the provided content.

Batch Size: Not explicitly stated.

Number of GPUs: One A100 GPU per configuration (no multi-GPU setup mentioned for individual rows).

The benchmark did not disclose the precision format (FP32, FP16, INT8) used during testing.

11. user (2026-05-31T04:25:48.159Z)

Web search results for query: "PaddleOCR PP-StructureV3 actual GPU memory usage real world RTX 3060 4060 issue"

Links: [{"title":"PP-StructureV3 Introduction - PaddleOCR Documentation","url":"http://www.paddleocr.ai/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html"},{"title":"PaddleOCR/docs/version3.x/algorithm/PP-StructureV3/PP-StructureV3.en.md at main · PaddlePaddle/PaddleOCR","url":"https://github.com/PaddlePaddle/PaddleOCR/blob/main/docs/version3.x/algorithm/PP-StructureV3/PP-StructureV3.en.md"},{"title":"Githubusercontent","url":"https://raw.githubusercontent.com/PaddlePaddle/PaddleOCR/main/docs/version3.x/algorithm/PP-StructureV3/PP-StructureV3.en.md"},{"title":"PaddleOCR-VL : Best OCR AI model. How to use PaddleOCR-VL for free? | by Mehul Gupta | Data Science in Your Pocket | Medium","url":"https://medium.com/data-science-in-your-pocket/paddleocr-vl-best-ocr-ai-model-e15d9e37a833"},{"title":"How to use GPU in paddleocr · Issue #10429 · PaddlePaddle/PaddleOCR","url":"https://github.com/PaddlePaddle/PaddleOCR/issues/10429"},{"title":"The Best Open-Source and Open-Weight AI Models for OCR | MerginIT Blog","url":"https://merginit.com/blog/15072025-best-ocr-ai-models"},{"title":"GitHub - PaddlePaddle/PaddleOCR: Turn any PDF or image document into structured data for your AI. A powerful, lightweight OCR toolkit that bridges the gap between images/PDFs and LLMs. Supports 100+ languages. · GitHub","url":"https://github.com/PaddlePaddle/PaddleOCR"},{"title":"Is it possible to train ppocr_v4 on a 6GB GPU? I am running out of memory · Issue #10783 · PaddlePaddle/PaddleOCR","url":"https://github.com/PaddlePaddle/PaddleOCR/issues/10783"}]

Based on the search results, I found some relevant information about PP-StructureV3 and GPU memory usage, though the results don't contain specific details about RTX 3060 or 4060 GPU memory issues.

Key Findings:

Official testing of PP-StructureV3 was conducted with V100 and A100 GPUs using CUDA 11.8 and cuDNN 8.9, which are high-end professional GPUs rather than consumer-grade hardware like the RTX 3060 or 4060.

Consumer-grade GPUs like NVIDIA RTX 3060 and RTX 4060 with 8GB-12GB VRAM can often run smaller OCR models, though this refers to models like TrOCR, Donut, and smaller vision-language models rather than the full PP-StructureV3 pipeline.

While PP-StructureV3 is recommended to run on GPU for optimal performance, it also supports CPU inference, which may be a viable alternative if GPU memory becomes a bottleneck.

Note: The official PaddleOCR documentation and benchmarks don't appear to include specific memory usage data for the RTX 3060 or RTX 4060 GPUs. The published performance metrics focus on professional-grade hardware (V100, A100). For real-world memory usage on consumer GPUs, you may need to consult community forums or GitHub issues specifically discussing this issue.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. assistant (2026-05-31T04:25:52.488Z)

The numbers are confirmed precisely against the primary source. Now let me assess the inferential leap. The claim says peak 7.4 GB on A100 makes it "feasible on consumer 8-12 GB NVIDIA GPUs." Let me check what precision PaddleOCR uses by default (FP32 vs FP16), and the CUDA version, since that affects whether the same 7.4 GB number transfers to a consumer card. There are also notes the benchmark used CUDA 11.8/cuDNN 8.9. Let me get the exact benchmark preamble.

13. user (2026-05-31T04:26:01.577Z)

PP-StructureV3 Benchmark Information

Software Environment

The pipeline benchmark section states: "PaddlePaddle 3.1.0?CUDA 11.8?cuDNN 8.9" with "PaddleX @ develop (f1eb28e23cfa54ce3e9234d2e61fcb87c93cf407)."

Precision/data type is **not specified** in the document.

Per-Image/Page Metrics

The benchmark data is "per image" ? the test strategy notes "Test data containing 280 images" and metrics are computed across this dataset.

Table Column Headers

The expandable pipeline benchmark table contains these columns:
- Pipeline configuration
- Hardware
- Avg. inference time (s)
- Peak CPU utilization (%)
- Avg. CPU utilization (%)
- Peak host memory (MB)
- Avg. host memory (MB)
- Peak GPU utilization (%)
- Avg. GPU utilization (%)
- Peak device memory (MB)
- Avg. device memory (MB)

Lightweight Configuration Models

The document does **not explicitly state which specific models the lightweight configuration uses**. It only labels configurations as "PP_StructureV3-lightweight" without detailing model names.

Specific Metrics for Intel 8350C + A100 (Lightweight)

- **Avg. inference time:** 0.61 seconds
- **Peak VRAM:** 7456.00 MB

14. user (2026-05-31T04:26:04.066Z)

Web search results for query: "PaddleOCR benchmark FP32 precision GPU memory CUDA 11.8 cuDNN 8.9 high performance inference"

Links: [{"title":"Put to Test: PaddleOCR Engine Example and Benchmark","url":"https://converter.app/blog/paddleocr-engine-example-and-benchmark"},{"title":"High-Performance Inference - PaddleOCR Documentation","url":"https://paddlepaddle.github.io/PaddleOCR/main/en/version3.x/deployment/high_performance_inference.html"},{"title":"[GPU OOM] ?High-Performance Inference? OOM during model initialization ? PaddlePaddle 3.0.0 + PaddleOCR 3.0.1 + CUDA 11.8 cuDNN 8.9 TRT 8.6 · Issue #73435 · PaddlePaddle/Paddle","url":"https://github.com/paddlepaddle/paddle/issues/73435"},{"title":"PaddlePaddle Release 24.06 - NVIDIA Docs","url":"https://docs.nvidia.com/deeplearning/frameworks/paddle-paddle-release-notes/rel-24-06.html"},{"title":"PaddlePaddle Release 24.09 - NVIDIA Docs","url":"https://docs.nvidia.com/deeplearning/frameworks/paddle-paddle-release-notes/rel-24-09.html"},{"title":"PaddleOCR-VL-1.5: Towards a Multi-Task 0.9B VLM for Robust In-the-Wild Document Parsing","url":"https://arxiv.org/pdf/2601.21957"},{"title":"PaddleOCR-VL: Boosting Multilingual Document Parsing via a 0.9B Ultra-Compact Vision-Language Model","url":"https://arxiv.org/pdf/2510.14528"},{"title":"PaddlePaddle Release 24.04 - NVIDIA Docs","url":"https://docs.nvidia.com/deeplearning/frameworks/paddle-paddle-release-notes/rel-24-04.html"}]

I've completed the web search for your query about PaddleOCR benchmarks with FP32 precision, GPU memory, and CUDA 11.8/cuDNN 8.9 specifications. Here are the key findings:

Key Results:

High-Performance Inference Support:
PaddleOCR supports CUDA and cuDNN versions compatible with CUDA 11.8 + cuDNN 8.9 or CUDA 12.6 + cuDNN 9.5. PaddleOCR only supports TensorRT 8.6.1.6 in the CUDA 11.8 environment.

Performance Characteristics:
The search results show that PaddleOCR with GPU processed images in 2.07 seconds per image, which is a 46% performance increase compared to CPU processing.

GPU Memory Considerations:
One documented issue reveals GPU memory requirements during high-performance inference mode, where users encountered cases where 196.875 MB needed to be allocated, with 11.609 GB already allocated on a GPU.

FP32 Precision Support:
PaddlePaddle supports FP32 (default) scripts with AMP enabling FP16 operations that provide 2X reduced memory bandwidth and storage for intermediates, though this represents mixed precision rather than FP32-only benchmarks.

The official PaddleOCR documentation confirms support for the exact CUDA 11.8 and cuDNN 8.9 versions you specified, particularly for high-performance inference scenarios.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. assistant (2026-05-31T04:26:08.223Z)

This is a critical finding. The benchmark uses CUDA 11.8 + cuDNN 8.9 + PaddleX @ develop, which corresponds to PaddleOCR's "high-performance inference" mode (which typically uses TensorRT/Paddle Inference optimizations and FP32 by default per the search result). There's also a documented GPU OOM issue (#73435) on exactly this software stack during model initialization. Let me look at that OOM issue and check whether the "lightweight" benchmark numbers reflect high-performance inference mode (which can require MORE memory during TensorRT engine building, not less).

16. user (2026-05-31T04:26:14.426Z)

GitHub Issue Summary

GPU Hardware:
The user had an NVIDIA GeForce RTX 3060 graphics card with 12GB total VRAM.

Out-of-Memory Error:
During PaddleOCR 3.0.1 High-Performance Inference initialization, the system attempted to allocate 196.875 MB but failed. The error message stated: "Cannot allocate 196.875 MB on GPU 0. A total of 11.609 GB is already allocated, leaving only 157.063 MB available."

Context:
The memory exhaustion occurred during model format conversion and engine selection?before actual inference processing began. The user confirmed batch_size was set to 1 and verified no competing GPU processes existed, suggesting the framework itself consumed excessive memory during initialization.

The issue does not specify whether default or lightweight models were being used, only that High-Performance Inference mode (`--enable_hpi True`) was enabled with PaddlePaddle 3.0.0, CUDA 11.8, and cuDNN 8.9.

17. user (2026-05-31T04:26:18.912Z)

Web search results for query: ""PP-StructureV3" benchmark "high-performance inference" TensorRT lightweight device memory"

Links: [{"title":"PP-StructureV3 Introduction - PaddleOCR Documentation","url":"https://paddlepaddle.github.io/PaddleOCR/main/en/version3.x/algorithm/PP-StructureV3/PP-StructureV3.html"},{"title":"Accelerating Deep Learning Inference: A Comparative ...","url":"https://userweb.cs.txstate.edu/~k_y47/webpage/pubs/electronics25.pdf"},{"title":"Benchmarking Default Performance ? TensorRT-LLM","url":"https://nvidia.github.io/TensorRT-LLM/performance/performance-tuning-guide/benchmarking-default-performance.html"},{"title":"Best Practices ? NVIDIA TensorRT","url":"https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html"},{"title":"LLM Inference Benchmarking: Performance Tuning with TensorRT-LLM | NVIDIA Technical Blog","url":"https://developer.nvidia.com/blog/llm-inference-benchmarking-performance-tuning-with-tensorrt-llm"},{"title":"Accelerating Deep Learning Inference: A Comparative Analysis of Modern Acceleration Frameworks","url":"https://www.mdpi.com/2079-9292/14/15/2977"},{"title":"TensorRT SDK | NVIDIA Developer","url":"https://developer.nvidia.com/tensorrt"}]

Based on the search results, I found relevant information about PP-StructureV3 and TensorRT inference optimization:

PP-StructureV3 Overview

Different model selections for PP-StructureV3 affect GPU memory consumption and inference speed, allowing users to choose an appropriate model based on their device capabilities and sample complexity. While PP-StructureV3 is recommended to run on GPU for optimal performance, it also supports CPU inference, with lightweight configurations available for CPU-only environments.

Benchmark Information

Performance testing of PP-StructureV3 under different GPU environments was conducted on V100 and

A100 GPUs, with requirements including Paddle 3.0, PaddleOCR 3.0.0, CUDA 11.8, and cuDNN 8.9. Additional serving inference testing was performed on an NVIDIA A100 with Intel Xeon Platinum 8350C environment.

TensorRT Performance Characteristics

Regarding TensorRT's performance for high-performance inference: TensorRT delivers the highest inference performance with batch-tuned engines and hardware-aware optimizations, achieving the highest throughput across most models. TensorRT is an ecosystem of tools that includes inference compilers, runtimes, and model optimizations delivering low latency and high throughput for production applications.

The search results confirm that PP-StructureV3 offers flexible model configurations for managing device memory constraints, and TensorRT is recognized as a leading framework for high-performance inference optimization.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. assistant (2026-05-31T04:26:19.322Z)

You've hit your session limit · resets 1:30pm (Asia/Calcutta)